

Week 5 Benchmark Report

Student ID: 112006265

Name: 蔡定宇

HPL

I built and ran HPL using MPI and OpenBLAS. I changed the `HPL.dat` parameters to:

- $N = 1000$
- $NB = 64$
- $P = 2$
- $Q = 2$

```
william@head:~/hpl/bin/linux$ cat HPL.dat
HPLinpack benchmark input file
Innovative Computing Laboratory, University of Tennessee
HPL.out      output file name (if any)
6            device out (6=stdout,7=stderr,file)
1            # of problems sizes (N)
1000         Ns
1            # of NBs
64           NBs
0            PMAP process mapping (0=Row-,1=Column-major)
1            # of process grids (P x Q)
2           Ps
2           Qs
16.0         threshold
3            # of panel fact
0 1 2        PFACTs (0=left, 1=Crout, 2=Right)
2            # of recursive stopping criterium
2 4          NBMINs (>= 1)
1            # of panels in recursion
2            NDIVs
3            # of recursive panel fact.
0 1 2        RFACTs (0=left, 1=Crout, 2=Right)
1            # of broadcast
0            BCASTs (0=1rg,1=1rM,2=2rg,3=2rM,4=Lng,5=LnM)
1            # of lookahead depth
0            DEPTHS (>=0)
2            SWAP (0=bin-exch,1=long,2=mix)
64           swapping threshold
0            L1 in (0=transposed,1=no-transposed) form
0            U  in (0=transposed,1=no-transposed) form
1            Equilibration (0=no,1=yes)
8            memory alignment in double (> 0)
william@head:~/hpl/bin/linux$
```

I changed these values because the assignment requires different parameters from the presentation. N is the matrix size, NB is the block size, and $P \times Q$ is the process grid. Since I used 4 MPI processes, I chose a balanced 2×2 grid. The block size 64 is also within the recommended range from the slides.

Run command:

```
mpirun -np 4 --host head:2,work1:2 --map-by ppr:2:node ./xhpl > output.txt
```

Week 5 Benchmark Report

Student ID: 112006265

Name: 蔡定宇

```
william@head:~/hpl/bin/linux$ tail -n 40 output.txt
=====
T/V          N    NB    P    Q          Time          Gflops
=====
WR00R2C4      1000    64     2     2          0.40          1.6676e+00
HPL_pdgesv() start time Fri Apr  3 15:22:03 2026

HPL_pdgesv() end time   Fri Apr  3 15:22:03 2026

=====
||Ax-b||_oo/(eps*(||A||_oo*||x||_oo+||b||_oo)*N)= 5.20225868e-03 ..... PASSED
=====
T/V          N    NB    P    Q          Time          Gflops
=====
WR00R2R2      1000    64     2     2          0.43          1.5469e+00
HPL_pdgesv() start time Fri Apr  3 15:22:03 2026

HPL_pdgesv() end time   Fri Apr  3 15:22:04 2026

=====
||Ax-b||_oo/(eps*(||A||_oo*||x||_oo+||b||_oo)*N)= 4.49164087e-03 ..... PASSED
=====
T/V          N    NB    P    Q          Time          Gflops
=====
WR00R2R4      1000    64     2     2          0.42          1.5943e+00
HPL_pdgesv() start time Fri Apr  3 15:22:04 2026

HPL_pdgesv() end time   Fri Apr  3 15:22:04 2026

=====
||Ax-b||_oo/(eps*(||A||_oo*||x||_oo+||b||_oo)*N)= 4.23689109e-03 ..... PASSED
=====

Finished      18 tests with the following results:
              18 tests completed and passed residual checks,
               0 tests completed and failed residual checks,
               0 tests skipped because of illegal input values.

=====

End of Tests.
=====
william@head:~/hpl/bin/linux$
```

The run completed successfully and showed PASSED. Example Gflops results were:

- 1.6676e+00
- 1.5469e+00
- 1.5943e+00

The final summary showed that all 18 tests passed.

HPCG

I built and ran HPCG with the following modified parameters:

- 56 56 56
- 15

Week 5 Benchmark Report

Student ID: 112006265

Name: 蔡定宇

```
william@head:~/hpcg/build/bin$ cat hpcg.dat
HPCG benchmark input file
Sandia National Laboratories; University of Tennessee, Knoxville
56 56 56
15
william@head:~/hpcg/build/bin$
```

I changed them because the assignment requires values different from the presentation. These three numbers are the problem dimensions, and 15 is the runtime in seconds. I chose 56 56 56 because they are multiples of 8 and valid for the benchmark.

Run command:

```
mpirun -np 4 --host head:2,work1:2 --map-by ppr:2:node ./xhpcg
```

```
william@head:~/hpcg/build/bin$ tail -n 40 HPCG-Benchmark_3.1_2026-04-03_15-36-22.txt
Benchmark Time Summary::WAXPBY=0.140947
Benchmark Time Summary::SpMV=2.68084
Benchmark Time Summary::MG=14.5605
Benchmark Time Summary::Total=17.8957
Floating Point Operations Summary=
Floating Point Operations Summary::Raw DDOT=1.06072e+09
Floating Point Operations Summary::Raw WAXPBY=1.06072e+09
Floating Point Operations Summary::Raw SpMV=9.44433e+09
Floating Point Operations Summary::Raw MG=5.26475e+10
Floating Point Operations Summary::Total=6.42133e+10
Floating Point Operations Summary::Total with convergence overhead=6.42133e+10
GB/s Summary=
GB/s Summary::Raw Read B/W=22.1152
GB/s Summary::Raw Write B/W=5.11122
GB/s Summary::Raw Total B/W=27.2264
GB/s Summary::Total with convergence and optimization phase overhead=27.0414
GFLOP/s Summary=
GFLOP/s Summary::Raw DDOT=2.07215
GFLOP/s Summary::Raw WAXPBY=7.52565
GFLOP/s Summary::Raw SpMV=3.5229
GFLOP/s Summary::Raw MG=3.61577
GFLOP/s Summary::Raw Total=3.58821
GFLOP/s Summary::Total with convergence overhead=3.58821
GFLOP/s Summary::Total with convergence and optimization phase overhead=3.56383
User Optimization Overheads=
User Optimization Overheads::Optimization phase time (sec)=3.2e-07
User Optimization Overheads::Optimization phase time vs reference SpMV+MG time=4.63603e-06
DDOT Timing Variations=
DDOT Timing Variations::Min DDOT MPI_Allreduce time=0.199069
DDOT Timing Variations::Max DDOT MPI_Allreduce time=0.32629
DDOT Timing Variations::Avg DDOT MPI_Allreduce time=0.278221
Final Summary=
Final Summary::HPCG result is VALID with a GFLOP/s rating of=3.56383
Final Summary::HPCG 2.4 rating for historical reasons is=3.58821
Final Summary::Reference version of ComputeDotProduct used=Performance results are most likely suboptimal
Final Summary::Reference version of ComputeSPMV used=Performance results are most likely suboptimal
Final Summary::Reference version of ComputeMG used=Performance results are most likely suboptimal
Final Summary::Reference version of ComputeWAXPBY used=Performance results are most likely suboptimal
Final Summary::Results are valid but execution time (sec) is=17.8957
Final Summary::Official results execution time (sec) must be at least=1800
william@head:~/hpcg/build/bin$
```

The result file showed:

Week 5 Benchmark Report

Student ID: 112006265

Name: 蔡定宇

- HPCG result is VALID
- GFLOP/s rating = 3.56383

STREAM

STREAM measures main memory bandwidth. I changed the array size from the presentation example to:

- `STREAM_ARRAY_SIZE = 60000000`

```
william@head:~/STREAM$ cat output.txt
=====
STREAM version $Revision: 5.10 $
=====
This system uses 8 bytes per array element.
=====
Array size = 60000000 (elements), Offset = 0 (elements)
Memory per array = 457.8 MiB (= 0.4 GiB).
Total memory required = 1373.3 MiB (= 1.3 GiB).
Each kernel will be executed 30 times.
The *best* time for each kernel (excluding the first iteration)
will be used to compute the reported bandwidth.
=====
Number of Threads requested = 2
Number of Threads counted = 2
=====
Your clock granularity/precision appears to be 1 microseconds.
Each test below will take on the order of 26960 microseconds.
(= 26960 clock ticks)
Increase the size of the arrays if this shows that
you are not getting at least 20 clock ticks per test.
=====
WARNING -- The above is only a rough guideline.
For best results, please be sure you know the
precision of your system timer.
=====
Function      Best Rate MB/s  Avg time     Min time     Max time
Copy:          34579.7    0.028228    0.027762    0.029138
Scale:         20915.4    0.046033    0.045899    0.046480
Add:           22085.9    0.065691    0.065200    0.066718
Triad:         22130.8    0.065510    0.065068    0.067840
=====
Solution Validates: avg error less than 1.000000e-13 on all three arrays
=====
william@head:~/STREAM$
```

I changed it because the assignment requires a different parameter. The array size is related to hardware because if the array is too small, the data may stay in cache. A large array is better for measuring real main-memory bandwidth.

Compile command:

```
make clean
make CC=gcc CFLAGS="-O3 -march=native -fopenmp -DSTREAM_ARRAY_SIZE=60000000 -
DNTIMES=30 -DSTREAM_TYPE=double" stream_c.exe
```

Week 5 Benchmark Report

Student ID: 112006265

Name: 蔡定宇

Run command:

```
./stream_c.exe
```

Results:

- Copy: 34579.7 MB/s
- Scale: 20915.4 MB/s
- Add: 22085.9 MB/s
- Triad: 22130.8 MB/s

The output ended with `Solution Validates.`

OSU

I used OSU to compare MPI communication on the same node and on different nodes.

Commands:

```
mpirun -np 2 --host head:2 ./osu_latency  
mpirun -np 2 --host head:2 ./osu_bw
```

```
mpirun -np 2 --host head:1,work1:1 ./osu_latency  
mpirun -np 2 --host head:1,work1:1 ./osu_bw
```

Same node (head:2):

Week 5 Benchmark Report

Student ID: 112006265

Name: 蔡定宇

```
william@head:~/osu-micro-benchmarks-7.5.2/c/mpi/pt2p
```

#	Size	Avg Latency(us)
1		0.16
2		0.16
4		0.15
8		0.15
16		0.17
32		0.16
64		0.16
128		0.17
256		0.20
512		0.27
1024		0.28
2048		0.38
4096		1.82
8192		2.15
16384		2.83
32768		4.09
65536		6.38
131072		11.58
262144		24.90
524288		43.94
1048576		85.98
2097152		170.53
4194304		347.09

```
william@head:~/osu-micro-benchmarks-7.5.2/c/mpi/pt2p
```

```
william@head:~/osu-micro-benchmarks-7.5.2/c/mpi/
```

#	Size	Bandwidth (MB/s)
1		16.46
2		31.10
4		62.30
8		112.59
16		259.29
32		482.72
64		742.54
128		981.64
256		1711.19
512		3706.83
1024		6414.28
2048		7769.30
4096		2781.21
8192		4363.36
16384		6332.78
32768		8700.52
65536		10700.47
131072		11903.16
262144		12426.07
524288		12127.94
1048576		12286.70
2097152		12409.57
4194304		12355.23

```
william@head:~/osu-micro-benchmarks-7.5.2/c/mpi/
```

- 1-byte latency: 0.16 us
- peak bandwidth: 12426.07 MB/s
- 4 MiB latency: 347.09 us

Different nodes (head:1,work1:1):

Week 5 Benchmark Report

Student ID: 112006265

Name: 蔡定宇

<pre>william@head:~/osu-micro-benchmarks-7.5.2/c/mpi/pt2</pre> <pre># OSU MPI Latency Test v7.5.2 # Datatype: MPI_CHAR. # Size Avg Latency(us) 1 23.95 2 23.69 4 23.74 8 23.90 16 23.66 32 23.77 64 23.64 128 23.90 256 24.35 512 23.86 1024 23.91 2048 29.98 4096 32.28 8192 35.68 16384 41.65 32768 50.10 65536 109.79 131072 124.79 262144 179.93 524288 230.63 1048576 340.68 2097152 566.27 4194304 1118.65 william@head:~/osu-micro-benchmarks-7.5.2/c/mpi/pt2</pre>	<pre>william@head:~/osu-micro-benchmarks-7.5.2/c/mpi/pt</pre> <pre># OSU MPI Bandwidth Test v7.5.2 # Datatype: MPI_CHAR. # Size Bandwidth (MB/s) 1 0.23 2 0.46 4 0.92 8 1.99 16 3.96 32 8.17 64 16.32 128 31.18 256 58.71 512 109.68 1024 212.80 2048 422.50 4096 796.99 8192 1350.11 16384 1862.21 32768 2618.37 65536 2598.97 131072 3164.62 262144 3511.53 524288 3712.33 1048576 3834.47 2097152 3853.56 4194304 3783.39 william@head:~/osu-micro-benchmarks-7.5.2/c/mpi/pt</pre>
---	--

- 1-byte latency: 23.95 us
- peak bandwidth: 3853.56 MB/s
- 4 MiB latency: 1118.65 us

The same-node result is faster because communication stays inside one machine. The different-node result is slower because data has to go through the network between nodes.

IOR

I used IOR with parameters different from the presentation:

```
mpirun -np 2 ./src/ior \
-w -r \
-t 2m \
-b 256m \
-F \
-o testfile
```

Week 5 Benchmark Report

Student ID: 112006265

Name: 蔡定宇

```
william@head:~/ior$ cat Output.txt
IOR-4.1.0-dev: MPI Coordinated Test of Parallel I/O
Began      : Fri Apr 3 15:52:55 2026
Command line : ./src/ior -w -r -t 2m -b 256m -F -o testfile
Machine     : Linux head
TestID      : 0
StartTime    : Fri Apr 3 15:52:55 2026
Path        : testfile.00000000
FS          : 29.8 GiB  Used FS: 44.8%  Inodes: 1.9 Mi  Used Inodes: 13.0%

Options:
api      : POSIX
apiVersion :
test filename : testfile
access    : file-per-process
type      : independent
segments  : 1
ordering in a file : sequential
ordering inter file : no tasks offsets
nodes     : 1
tasks     : 2
clients per node : 2
memoryBuffer : CPU
dataAccess : CPU
GPUDirect  : 0
repetitions : 1
xfer size  : 2 MiB
block size : 256 MiB
aggregate file size : 512 MiB

Results:
access  bw(MiB/s)  IOPS    Latency(s)  block(KiB)  xfer(KiB)  open(s)  wr/rd(s)  close(s)  total(s)  iter
-----
write   1916.50     958.34    0.002887    262144      2848.00    0.000039  0.267128  0.056414  0.267154  0
read    17164        8586     0.000229    262144      2848.00    0.000223  0.029817  0.000467  0.029830  0

Summary of all tests:
Operation Max(MiB)  Min(MiB)  Mean(MiB)  StdDev  Max(OPs)  Min(OPs)  Mean(OPs)  StdDev  Mean(s)  Stonewall(s)  Stonewall(MiB)  Test#  #Tasks  tPN  reps  fPP  reord  reordoff  reordrand  seed  segcnt  blksiz
write aggs 1916.50  1916.50  1916.50    0.00    958.25    958.25    958.25    0.00    0.26715  NA            NA      0      2  2    1  1    0      1      0  0      1  268435456  2897
152 512.0 POSIX 0 0.00 8581.97 8581.97 8581.97 0.00 0.02983 NA NA 0 2 2 1 1 0 1 0 0 1 268435456 2897
read 17163.94 17163.94 17163.94 0.00 8581.97 8581.97 8581.97 0.00 0.02983 NA NA 0 2 2 1 1 0 1 0 0 1 268435456 2897
152 512.0 POSIX 0 0.00 8581.97 8581.97 8581.97 0.00 0.02983 NA NA 0 2 2 1 1 0 1 0 0 1 268435456 2897
Finished : Fri Apr 3 15:52:55 2026
william@head:~/ior$
```

I changed `-t` and `-b` because the assignment requires different parameters. Here, `-t 2m` means 2 MiB transfer size, and `-b 256m` means each process writes a 256 MiB block. `-F` means file-per-process.

Results:

- write bandwidth: 1916.50 MiB/s
- read bandwidth: 17164 MiB/s

The read speed is much higher than the write speed, which likely shows the effect of **page cache**. This means some read data may have been served from memory instead of storage.

Conclusion

In this assignment, I tested HPL, HPCG, STREAM, OSU, and IOR to measure compute, memory, communication, and storage performance. By changing the parameters from the presentation examples, I was able to observe how each benchmark behaves under different settings. The results also show that different parts of the system can become the main bottleneck depending on the workload.